

Lessons from Building StackSpot AI: A Contextualized AI Coding Assistant

Gustavo Pinto
Zup Innovation & UFPA
Belém, PA, Brazil
gustavo.pinto@zup.com.br

Alberto de Souza
Zup Innovation
São Paulo, SP, Brazil
alberto.tavares@zup.com.br

Cleidson R. B. de Souza
UFPA
Belém, PA, Brazil
cleidson.desouza@acm.org

Tarcísio Gotto
Zup Innovation
São Paulo, SP, Brazil
tarcisio@zup.com.br

João Batista Neto
Zup Innovation
São Paulo, SP, Brazil
joao.neto@zup.com.br

Edward Monteiro
StackSpot
São Paulo, SP, Brazil
edward.monteiro@stackspot.com

ABSTRACT

With their exceptional natural language processing capabilities, tools based on Large Language Models (LLMs) like ChatGPT and Co-Pilot have swiftly become indispensable resources in the software developer's toolkit. While recent studies suggest the potential productivity gains these tools can unlock, users still encounter drawbacks, such as generic or incorrect answers. Additionally, the pursuit of improved responses often leads to extensive prompt engineering efforts, diverting valuable time from writing code that delivers actual value. To address these challenges, a new breed of tools, built atop LLMs, is emerging. These tools aim to mitigate drawbacks by employing techniques like fine-tuning or enriching user prompts with contextualized information.

In this paper, we delve into the lessons learned by a software development team venturing into the creation of such a contextualized LLM-based application, using retrieval-based techniques, called StackSpot AI. Over a four-month period, the team, despite lacking prior professional experience in LLM-based applications, built the product from scratch. Following the initial product release, we engaged with the development team responsible for the code generative components. Through interviews and analysis of the application's issue tracker, we uncover various intriguing challenges that teams working on LLM-based applications might encounter. For instance, we found three main group of lessons: LLM-based lessons, User-based lessons, and Technical lessons. By understanding these lessons, software development teams could become better prepared to build LLM-based applications.

CCS CONCEPTS

• **Software and its engineering** → **Software design tradeoffs.**

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE-SEIP '24, April 14–20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0501-4/24/04

<https://doi.org/10.1145/3639477.3639751>

KEYWORDS

LLM, LLM-based applications, LLM for code, LLM4code, Code LLMs, Challenges

ACM Reference Format:

Gustavo Pinto, Cleidson R. B. de Souza, João Batista Neto, Alberto de Souza, Tarcísio Gotto, and Edward Monteiro. 2024. Lessons from Building StackSpot AI: A Contextualized AI Coding Assistant. In *46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '24)*, April 14–20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3639477.3639751>

1 INTRODUCTION

Large-language models (LLMs), with their exceptional natural language processing capabilities, are expected to revolutionize various industries [13]. Their advanced text generation, translation, and emergent abilities have streamlined content creation and customer interaction [32]. Moreover, they have enabled more efficient data analysis, enhancing decision-making processes in fields such as finance [29] and healthcare [23].

In the realm of software engineering, recent advancements in LLMs have yielded transformative tools like GitHub Copilot and Amazon CodeWhisper [31]. Arguably, Copilot and similar tools have significantly bolstered software development productivity by providing code suggestions, detecting bugs, and automating repetitive tasks [9]. In general, AI-powered code assistants aim to streamline development workflows, reducing coding errors and fostering collaboration among developers, marking a potential pivotal shift in the landscape of software engineering [4, 10, 26].

Major technology companies have entered a race to create their own AI-powered code assistants. GitHub CoPilot debuted in 2021, followed by the releases of ChatGPT and Amazon CodeWhisperer in 2022. In early 2023, Replit introduced GhostWriter, and in the early fall of the same year, IBM Watsonx Code Assistant and Google Project IDX were launched. There are also open-source alternatives, such as CodeGeeX [33], which contribute to this competitive landscape. However, building an AI-powered code assistant is far from straightforward [9, 10]. It involves multiple phases and substantial computational power, often restricting smaller companies and organizations from developing such tools due to economic constraints [2].

To overcome these financial barriers, alternative approaches that do not necessitate extensive training or fine-tuning have been introduced. One such approach combines retrieval-based techniques

to identify representative data and enrich the LLM's prompt [15]. Additionally, these retrieval-based techniques often enhance the generated responses [14]. This improvement occurs because the data used to enrich the LLM's prompt can be more recent and accurate, making these techniques not only economical but also highly contextualized. By employing these retrieval inspired methods, companies and organizations can harness the benefits of state-of-the-art models.

While there are scientific evaluations assessing the potential advantages of retrieval-based techniques in enhancing Large Language Model responses [7, 12, 16], comprehensive reports on real-world products utilizing these methods are still lacking in the literature. Consequently, it remains uncertain, for instance, whether the development of LLM-based applications introduces distinct technical challenges for software development teams. To address this gap, our research posed the following question:

RQ: What challenges do software development teams encounter when creating LLM-based applications?

This study aimed to bridge this gap by investigating this overlooked aspect. We accomplished this by closely observing and engaging with software engineers responsible for developing StackSpot AI¹, an LLM-based AI coding assistant. Through developer interviews and analysis of the issue tracker data, we compiled a detailed list of 14 lessons learned while building StackSpot AI, categorized into three key domains: LLM-based lessons, User-based lessons, and Technical lessons. These lessons encompass both the challenges developers faced as well as the solutions they adopted to address these challenges. By sharing these invaluable insights, our goal is to enlighten other software development teams interested in constructing LLM-based applications, equipping them to anticipate and effectively mitigate similar challenges in their projects. In summary, we noted that building an AI-powered code assistant and other LLM-based applications requires a lot of experimentation, similar to a trial-and-error process, which suggests new reports about how to construct similar tools are important.

The rest of this paper is organized as follows. The next section motivates the reason why we decided to create a *contextualized* AI-based coding assistant. This is followed by section 3 where we briefly describe StackSpot AI, our AI coding assistant. Then, the next section describes the methodology we used to extract our lessons learned. After that, we describe the set of 14 lessons learned divided into three groups: LLM-based lessons, User-based lessons, and Technical Lessons. This is followed by our Discussion where we compare these lessons with the literature. The following sections describe related work and the limitations of our study. Finally, we present our conclusions and suggestions for future work.

2 WHY DOES ONE NEED A CONTEXTUALIZED CODING ASSISTANCE?

Large language models based tools such as ChatGPT have significantly enhanced software development practices. However, they encounter challenges when providing context-specific or definitive answers. Without a user's contextual information, these tools might

yield generic and less helpful responses. Acknowledging these constraints, we decided on developing our customized AI coding assistant, utilizing OpenAI's model as our foundation. Our initiative stemmed from the need to bridge the gap left by existing tools.

By tailoring our assistant to comprehend nuanced contexts and evolving developer requirements, we aimed to deliver precise, insightful, and adaptive solutions. This bespoke approach ensures that developers receive tailored assistance, refining their coding experience and maximizing productivity. Our goal with StackSpot AI is not only to address contextual complexities but also enhance the overall software development journey, making it more efficient and user-friendly. In particular, StackSpot AI aims to tackle the following challenges that persists in general language models.

- (1) **Inconsistency in the responses:** Large language models exhibit inconsistency in their responses [19], sometimes delivering accurate answers while at other times producing random or seemingly clueless information due to their lack of genuine understanding.
- (2) **Extensive Prompt Engineering Challenges:** Another issue with general-purpose coding assistants is the substantial reliance on prompt engineering. Recent research highlights the considerable challenge posed by this requirement, as even minor adjustments to the input prompt can yield significant variations in the output [3, 27]. This sensitivity further complicates the usability and reliability of these tools.
- (3) **Lack of Domain-Specific Knowledge:** While tools like CoPilot offer valuable insights into general programming concepts, they fall short when it comes to translating specific organizational requirements into code. These models lack awareness of an organization's unique domain, making it challenging to bridge the gap between high-level project specifications and actual code implementation². Consequently, developers using these tools often find themselves needing to extensively refactor the generated code to align it with project requirements, coding standards, and other contextual factors.

In light of these challenges, we recognized the need for a more specialized coding assistant—one that could ingest organization-specific information and deliver tailored robust solutions. To address this need, we embarked on the development of our coding assistant, which we named StackSpot AI.

3 STACKSPOT AI

StackSpot AI is a coding AI assistance developed by Zup Innovation³, a sizeable Brazilian tech company. In this section we provide an overview of StackSpot AI as a product (Section 3.1), then we describe the problem that StackSpot AI is trying to solve (Section 3.2), the architecture of the proposed solution (Section 3.3), our prompt execution approach 3.4 and the team who built the solution (Section 3.5).

3.1 StackSpot AI as ChatBot

Figure 1 depicts the interaction between developers and StackSpot AI and showcases the integration through a VS Code plugin. Initially,

¹At the moment of this writing, StackSpot AI is available in a pre-release form at <https://ai.stackspot.com/>

²Among the several existing LLMs for code only Tabnine (<https://www.tabnine.com>) allows this type of customization.

³<https://www.zup.com.br/>

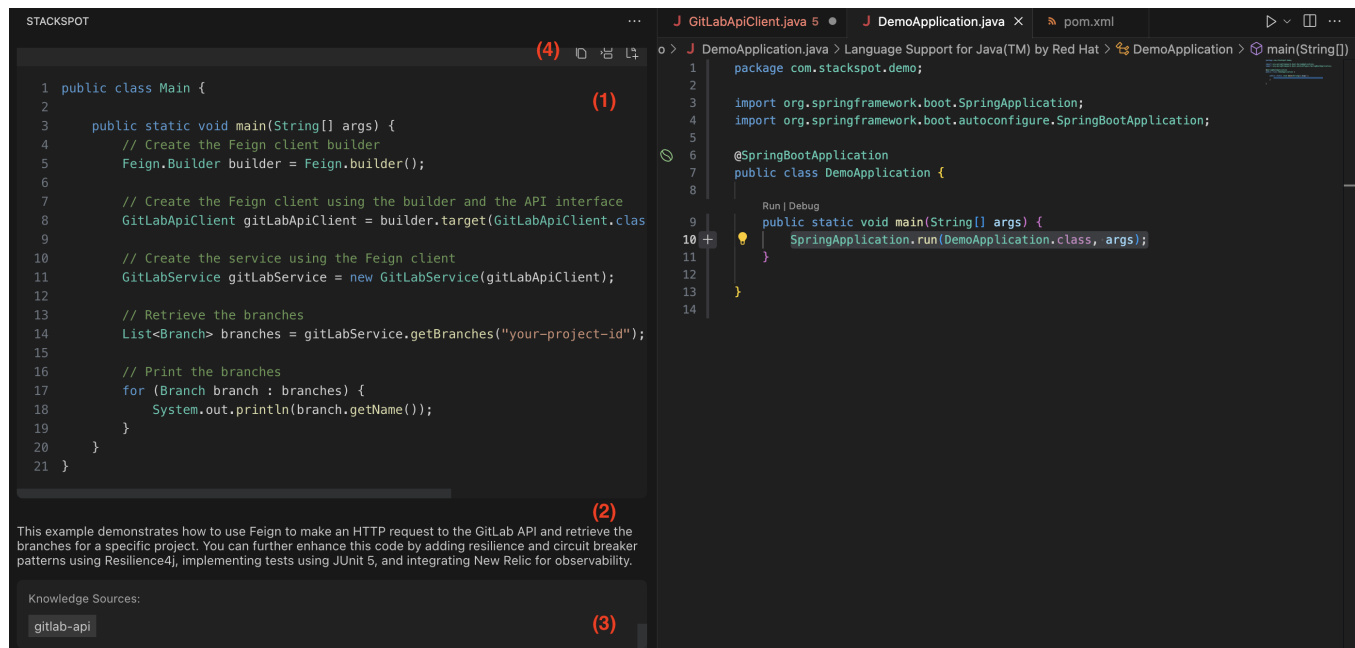


Figure 1: StackSpot AI's view in VS Code.

developers engage with StackSpot AI via the input field located at the bottom left. StackSpot AI responds to developers' queries on the left-hand side. Developers have the flexibility to pose multiple questions, albeit one at a time. Upon receiving a response ((1) in the Figure 1), StackSpot AI also explains the rationale for the generated code ((2) in the Figure 1). As we shall see in Section 3.2, StackSpot AI is intricately designed within a Retrieval-Augmented Generation (RAG) architecture [15]. Consequently, for every user query, StackSpot AI tries to locate representative documents, referred to as *knowledge sources*, enriching the user's prompt. Notably, within the plugin interface, users can view the selected knowledge source employed in shaping the prompt, enhancing transparency and user understanding ((3) in the Figure 1). Finally, if the developer believes the proposed code snippet is accurate, they can accept the suggestion using shortcut items in the IDE ((4) in the Figure 1), allowing StackSpot AI to seamlessly insert the generated code into an existing file.

3.2 StackSpot AI's idea

The main idea behind StackSpot AI is that we could generate better responses *if* we could enrich the prompt with contextualized information, that is, our *knowledge sources*. Such an enriching process is technically called Retrieval-Augmented Generation (RAG) [15].

RAG is an approach designed to enhance LLM-generated content by anchoring it in external knowledge sources. In question-answering systems, RAG accesses up-to-date, reliable information and provides transparency to users regarding the model's information sources, promoting trust and verifiability. Thus, RAG not only improves response quality but also mitigates the risk of sensitive data leakage and misinformation generation. An illustrative list of knowledge sources encompasses several items:

- (1) An extensive catalogue of APIs recurrently harnessed by the development team;
- (2) Exemplary code snippets serving for discerning coding paradigms or facilitating code modernization activities;
- (3) Customized artifacts written in natural language, including but not limited to, guidelines delineating the protocol for repository commits and a comprehensive list of software requirements to be implemented.

By providing relevant and up-to-date information to the LLM, RAG also reduces the need for constant model retraining and parameter updates, lowering computational and financial overhead [15], since there is no need to build a new foundation model or retrain an existing one.

To implement a RAG, we need to break it down into two main components: the *retrieval* component and the *generative* component.

- **Retrieval component:** This component is designed to fetch relevant information from a specific dataset. These components typically use information retrieval techniques or semantic search to identify the most relevant data for a given query. On the one hand, they are efficient at retrieving relevant information but at the other hand they are not capable of generating new content.
- **Generative component:** Generative components are designed to generate new content based on a specific prompt. Large Language Models excel at this type of task. To build StackSpot AI, we employed as the generative component the gpt-3.5-4k model, provided by OpenAI.

In summary, the core concept behind StackSpot AI revolves around using RAG to enhance response quality by enriching prompts with contextualized information. RAG integrates external knowledge

sources, ensuring the generated content is based on reliable information. In chat-based question-answering systems, RAG not only improves response quality but also establishes transparency by revealing the model's information sources, building user trust.

3.3 StackSpot AI's core components

Our specialized coding assistant StackSpot AI is built on top of OpenAI, using the `gpt-3.5-4k` model. The decision to opt for this model, rather than, for instance, `gpt-4` or `gpt-3.5-16k`, is rooted in a twofold rationale. Firstly, `gpt-4` significantly escalates token costs compared to `gpt-3.5-4k`, despite delivering enhanced responses. This cost disparity could prove prohibitive, especially for a product in its early stages, still seeking its market foothold. Secondly, our choice to maintain a compact token window (4k as opposed to 16k) stemmed from a few experiments we conducted. These experiments unveiled a decline in response quality, notably when dealing with extensive knowledge sources. Thus, we deliberated in terms of costs and response quality.

To manage the LLM workflow, we used the LangChain framework⁴. LangChain is an open-source framework designed to simplify the creation of applications using LLMs. It provides a standard interface for chains, integrations with other tools, and end-to-end chains for common applications. LangChain helped the team to ease the manipulation of OpenAI APIs, but also to take advantage of additional features, such as handling long conversations or employing additional LLMs.

The core of StackSpot AI has four main components: 1) the knowledge source component, 2) the RAG component, 3) the GPT component, and 3) the feedback loop component. We describe each one of them next.

- **Knowledge Source Component:** The Knowledge Source Component serves as the foundation of the architecture. It interfaces with diverse data sources, such as a catalogue of OpenAPIs, code snippets, or custom objects. This component employs retrieval models using semantic search algorithms to ensure accurate and contextually relevant data retrieval based on user queries. Extracted data becomes the basis for subsequent processing, ensuring a robust pool of context for the RAG system.
- **RAG Component:** The RAG module combines the strengths of the retrieval and generative components. In the retrieval phase, the module identifies the most relevant pieces of information from the retrieved data. These pieces of information serve as the context for the GPT Component. The generative model, often based on architectures like GPT (Generative Pre-trained Transformer), then utilizes this context to generate coherent and contextually appropriate responses.
- **GPT Component:** The GPT Component is the generative powerhouse of the architecture. Built upon the pre-trained language model `gpt-3.5-4k`, it utilizes the context provided by the RAG Component to craft coherent and contextual responses in natural language. The generated responses are provided back to StackSpot AI users.

- **Feedback Loop component:** StackSpot AI incorporates a feedback loop where user interactions and feedback are collected. For instance, when a user finds a response useful, it could 'like' the response; it could also 'dislike' otherwise.

StackSpot AI was also designed with scalability in mind. Components are containerized for easy deployment and management. Cloud-based solutions and container orchestration tools can be employed to ensure seamless scalability based on demand.

Last, but not least, it is important to mention that, while constructing StackSpot AI, we conducted a few informal user studies. By informal, we mean for instance, inviting software developers to quickly use the tool and then interview them about their experience using StackSpot AI. For instance, as we will discuss later, we explored different sizes of the conversation history. We also conducted a more controlled user study that is reported here [21].

3.4 StackSpot AI's Prompt Execution

In this work, we utilized OpenAI through its API⁵. We used the `gpt-3.5-4k` model with the temperature defined to zero. The temperature is a hyperparameter that affects the probability of token distributions in the model. It is possible to adjust the temperature to control the diversity and creativity of the generated texts. The temperature value ranges from 0 to 1. While a high temperature (e.g., 0.7+) leads to more diverse results, a low temperature (e.g., 0.2) tends to produce more deterministic results. The value of zero was chosen as an attempt to obtain similar answers based on similar questions.

3.5 StackSpot AI's Team

The software development team comprises ten engineers: one CTO, one product manager, two senior UX designers, one IDE designer, and five senior backend engineers. The team works in a distributed, remote-first manner. For communication, the team uses Discord, with text, audio, and video channels. Asynchronous communication is often preferred over synchronous ones, although the team meets synchronously for at least ~1 hour every weekday.

Although the team has a long track record in developing and deploying working software products, the team did not have previous professional experience in creating LLM-based applications. The team started StackSpot AI's development on April 2023 from scratch. During this timeframe the team was able to build a working version of StackSpot AI.

4 METHODOLOGY

In this research, we sought to understand the challenges in the development process of a LLM-based application and highlight the difficulties faced by developers during the development cycle. To achieve this goal, we started by 1) asking the developers their perception about the main challenges they encountered when writing such kind of applications. We then 2) mined issues in the repository that could offer additional insights on the developers' perception. We detail these items next.

⁴<https://python.langchain.com/>

⁵<https://platform.openai.com/docs/api-reference>

4.1 Asking developers' perception

To understand the developers' perception, we asked them two questions:

Question 1: I am trying to summarize a few lessons learned from building StackSpot AI. Can you list some technical challenges you faced when building StackSpot AI's core components?

Understanding the technical challenges encountered during the development of StackSpot AI, especially in the context of Large Language Models (LLMs), is crucial for drawing meaningful insights. By focusing on LLM-related hurdles, we can gain an understanding of the intricacies of AI-powered coding tools, enabling us to distill best practices that can be used by others creating LLM-based solutions.

Question 2: How did you figure out how to overcome the technical challenge?

Understanding how StackSpot AI's developers approach these obstacles could provide insights into their problem-solving skills and adaptability.

Although the StackSpot AI's team is comprised by ten engineers, these questions were sent only to the four developers that participated in the creation of the StackSpot AI's code component (see Section 3.3). One of the authors of this study is also part of the development team. We sent these messages through the company's internal chat system. If needed, we asked clarification questions to the team. Developer responses were coded using traditional qualitative research methods [5]. These responses provided valuable insights into practical challenges and creative solutions adopted during the process of developing an AI-based coding assistant.

4.2 Mining Issues Log

In our research methodology, we also manually mined GitHub issues to unveil the intricate challenges associated with constructing LLM-based applications. We analyzed a set of 74 issues opened on StackSpot AI's issue tracking platform. By studying the issues' description and the developers' interaction within these issues, we were able to complement the insights of the developers' hurdles. This exploration not only sheds some light on the existing obstacles but also paves the way for innovative solutions, helping to shape the future of LLM-driven applications.

4.3 Ethical Considerations

Throughout the research process, strict ethical standards were observed. Sensitive and personal information was treated with confidentiality, and the research was conducted in accordance with ethical principles established by the academic community.

5 LESSONS LEARNED

In this section, we curate a list of lessons learned while building StackSpot AI. We organize these lessons in three main groups: LLM-based lessons, User-based lessons, and Technical lessons. For each group of lessons, we provide concrete examples, with some explored solutions. Since our focus is on AI-based coding assistants, we will

provide more details for the LLM-based lessons, than User-based lessons, and last, the Technical lessons. The majority of these lessons came from the developers' perception. However, when appropriate, we complement their perception with the description of related issues found in the repositories.

5.1 LLM-based lessons

L1. Manage Tokens carefully: Effectively managing token limitations in LLM-generated responses is imperative due to the inherent constraints LLMs operate within. We found five issues related to this item. Some of the tasks in these issues included: "The platform continuously monitors prompt length and takes proactive measures to prevent overflow" and "Users receive a seamless and uninterrupted chat experience, with minimized interruptions due to prompt overflow." Notably, the total number of tokens utilized in a request encompasses both the user's prompt and the subsequent LLM response, calculated as $\text{total_tokens} = \text{tokens}(\text{prompt}) + \text{tokens}(\text{LLM_response})$. Anticipating the exact token count beforehand is challenging as setting a strict limit on LLM response tokens is unfeasible. Even instructing the LLM to provide an answer within a specific word limit does not guarantee deterministic results. Furthermore, the prompt's size naturally expands as we maintain a history of the last 20 user prompts and LLM responses (further detailed in lesson L5 **Manage Interaction History**). To address these complexities, the team explored diverse strategies. For instance, one approach was to set, say, a 10% threshold (considering the max token size, which is 4k). If $\text{tokens}(\text{prompt})$ plus 10% is greater than 4k, we could remove one of the oldest items of the history. Another approach encompassed dynamic summarization, where the system dynamically summarized longer prompts into concise, information-rich versions, optimizing token usage without compromising context. These strategies aimed to balance token economy with context retention, ensuring the generation of coherent and precise responses while adapting to the unpredictable nature of LLM processing. One can imagine that new LLMs with windows larger than 4k tokens might simplify this issue. However, this is not always the case with the Transformer architecture [25], because the longer the context, the higher the likelihood of the LLM to generate incorrect answers (e.g., [18]). Moreover, each programming language has different syntax elements that could complicate token management. In languages akin to Python, whitespace is integral to the language syntax. However, each whitespace character also accounts for a token. Consequently, optimization strategies applicable in some languages, such as eliminating all whitespace, cannot be universally applied across all programming languages. Each language's unique syntax and tokenization nuances necessitate tailored strategies, ensuring careful consideration before implementing optimizations.

L2. Break Knowledge Sources into chunks: One of the fundamental components in StackSpot AI's architecture is the retrieval component of RAG. To ensure the generative component of RAG delivers accurate responses, it requires robust contextual information, referred to in this paper as Knowledge Source (see section 3.3). A Knowledge Source can take various forms, ranging from source code to API catalogues and custom items written in natural language. However, due to limitations on the size of the generative component

(already detailed in Lesson L1), it is not feasible to include all available knowledge sources. For instance, using the `gpt-3.5-4k` model, we are constrained to 4k tokens. Consequently, the development team faced the challenge of chunking data effectively while still retrieving pertinent information. This task proved intricate due to the diverse nature of potential knowledge sources. There are four issues dedicated to deal with chunking of knowledge sources. Some of the tasks in these issues involve: “The team should suggest an appropriate approach for creating the Chunking Model for Event Schema, considering the unique characteristics of this Knowledge Source” and “Review the requirements and specifications for the wavelet-based chunk generation algorithm, understanding the objectives and expected outcomes”. While established techniques exist for chunking textual data (such as chunking by paragraph or using sentence-based approaches), there are fewer established options for APIs or code snippets. This happens due to the variety one could chunk; in the case of code snippets, one could chunk data based on lines of code, enclosed functions, classes, files, or encapsulated Abstract Syntax Tree (AST) subtrees. Each approach presents potential advantages and drawbacks, adding complexity to the decision-making process. At this moment, the team has not decided which approach works best.

L3. Find representative knowledge sources: Chunking and storing knowledge sources constitute only one facet of the challenge; retrieving the most pertinent chunk of knowledge based on user prompts presents another. There are two issues related to this item. Some of the tasks include: “Ensure that the KS search considers the semantic context of the prompt for accurate results” and “The system searches for relevant Knowledge Sources that match the prompt’s context”. The complexity of these tasks arises from the need for StackSpot AI to locate knowledge sources, in various formats, related to user queries composed in natural language. To tackle this intricate relationship, our team explored diverse semantic search methods, including Euclidean distance, cosine distance, and max inner product. Simultaneously, we endeavoured to transform non-natural language knowledge sources into a natural language format. To achieve this, we employed `gpt-3.5-4k` to perform the translation. While the observed results did not show a discernible difference in response quality, the approach did enable a reduction in overall token count (see Lesson L1). This reduction was possible due to the elimination of elements such as brackets, parentheses, and whitespaces, each consuming a token. However, this translation method requires an additional request to OpenAI, but only when incorporating a new knowledge source into the database.

L4. Assess the quality of the response: In our initial implementation, we quickly realized that the quality of responses is directly correlated with the quality of the knowledge source employed. If the search mechanism selects irrelevant knowledge sources, the likelihood is high that the answer will not be directly aligned with the users’ requirements. Unlike traditional unit testing, evaluating the quality of an LLM answer is not a deterministic process. As highlighted by related works, subtle modifications in the prompt can yield different responses. To gauge response quality, the team employed conventional NLP metrics, including BLEU [20] and ROUGE [17], and explored recently introduced metrics from the

forefront of evaluating LLMs [8]. Our present understanding underscores the necessity for further research to derive robust metrics capable of addressing this intricate landscape. There is also one issue focused on this task, entitled: “Implementing a Benchmarking Tool to Assess Response Quality Post-Changes”. In this issue we were intended to evaluate response quality following changes in prompts, Knowledge Sources, and similarity search algorithms. The user story emphasizes the creation of robust metrics aligned with industry standards, integration with prompt and algorithm updates, exploration of benchmarking tools, user-friendly interfaces for insights, comprehensive documentation, and a continuous improvement mechanism based on benchmarking results. Testing and validation processes ensure the system’s effectiveness in accurately measuring response quality post-changes.

L5. Manage Interaction History: StackSpot AI, designed as a chatbot (see section 3.1), inherently prioritizes managing past conversations effectively. There are two issues related to managing the chat history, with tasks such as “The feature’s performance is optimized to handle the loading and display of chat history efficiently, even for users with a substantial chat history.” To accomplish this, we implemented a history mechanism. Initially, we tracked the last 40 messages, each comprising the user’s prompt *and* the corresponding LLM response. However, adhering to this 40-message limit swiftly led to prompt overflow, a situation arising from exceeding the token limit within a single LLM request (see Lesson L1). To address this, a simple solution emerged: reducing the size of the history. Our team conducted experiments with various sizes, ultimately determining that limiting the history to 20 messages proved to be the most effective approach. To manage these messages more effectively, the team decided to store them in a Dynamo database, offering scalability and the possibility to expunge old messages automatically (using the Time-To-Live feature). This adjustment helped to improve the management of conversation flow and token usage, similarly to what has recently been reported by Ross et al. [22].

L6. Strive for a Balance between Accuracy and Responsiveness: Building an AI assistant that provides contextualized assistance without extensive training of the LLM poses a significant challenge. This assistant needs to comprehend intricate contextual nuances, which means integrating external knowledge seamlessly, requiring innovative techniques to extract and blend relevant information from various sources. This challenge is scattered in different StackSpot AI domains. For instance, there is one issue that mentions the need to “[...] enhance the Knowledge Sources search by similarity to improve its effectiveness and relevance” while other issue is about “Ensuring the conversational agent and chat tools maintain a high level of accuracy and responsiveness, providing prompt and relevant code suggestions to the user”. Achieving a balance between depth of context and computational efficiency is crucial for creating a responsive and accurate AI assistant. Computational efficiency is important because the suggestions generated by the assistant should be presented in milliseconds. Organizations interested in creating their own AI-coding assistant should be aware of this trade-off between correctness (accuracy) and speed (responsiveness).

5.2 User-based lessons

L7. Ensure Good Responses: Developing a methodology that consistently generates high-quality responses from an LLM is an ongoing challenge. Note that this is more complex than prompt engineering [6] that focuses on the process of structuring words that can be interpreted and understood by the generative AI model. Crafting responses that align with user queries, context, and intent requires a delicate balance of linguistic finesse and technical accuracy. There is one issue regarding this problem, with tasks such as: “When the LLM generates responses, the Knowledge Objects utilized to generate each response should be displayed in the IDE extension along with the answers provided” and “The displayed Knowledge Objects should be relevant and accurately reflect the information used to generate the corresponding responses.” This multifaceted approach involves continuous experimentation and an in-depth understanding of the natural language. The objective is to enhance not just the correctness, but also the naturalness and relevance of the generated responses, ensuring a more seamless and intuitive user experience.

L8. Be aware of Complex Development Contexts: Grasping multifaceted software development contexts demands a nuanced approach. Handling diverse programming languages, frameworks, and project requirements requires deep semantic analysis. Building a knowledge base that comprehensively captures these complexities, enabling the AI to respond accurately and contextually to intricate queries, is pivotal. Deepening the AI’s understanding of intricate contexts enhances its capability to assist developers effectively across a wide spectrum of development scenarios.

L9. Gather Feedback, Learn and Adapt to it: The iterative process of collecting user feedback and incorporating it into the system’s learning loop is fundamental. Therefore, building a robust mechanism to process diverse user inputs, analyze feedback effectively, and adjust the model’s behavior accordingly is essential. There are four issues regarding the use of metrics to assess StackSpot AI usage. As described in one of the issues “I want the ability to provide feedback on the answers generated by the Language Model (LLM) directly within the integrated development environment (IDE). This feedback mechanism will allow me to express my satisfaction or dissatisfaction with the responses, helping StackSpot AI improve the quality of answers over time.” By creating and monitoring such metrics could help in refining the AI’s responses, making them contextually relevant and reflective of user preferences, thereby enhancing the overall user experience.

L10. Allow Customization: Tailoring the AI experience to meet the diverse needs of individual developers necessitates building adaptable systems. Designing customizable interfaces, user-specific preferences, and modular components empowers users to personalize their interactions with the StackSpot AI. Striking a balance between flexibility and ease of use, ensuring users can efficiently customize their experience, is pivotal in creating a user-centric and adaptable AI coding assistant.

L11. Design allowing the Optimal Utilization of the Created Concepts: StackSpot AI requires users to get acquainted with concepts, such as knowledge sources and workspaces (which simulates a work environment) that they might not be aware of. Maximizing the utility

of these concepts is paramount in order to enhance the user experience. Creating intuitive workflows, seamless integration points, and intuitive user interfaces ensures that these concepts enhance the user experience organically, aligning with the user’s workflow and providing value at every interaction point.

5.3 Technical lessons

L12. Balance Scalability and Latency: Ensuring the scalability of StackSpot AI to handle a substantial volume of concurrent users while maintaining low latency is a multifaceted challenge. It necessitates optimizing infrastructure, utilizing distributed computing, and implementing load-balancing mechanisms. Balancing response time with system load ensures a seamless user experience, even during peak usage, making scalability a paramount concern in the system’s architectural design.

L13. Be Sensitive to Data Management: Safeguarding sensitive user data and proprietary code snippets is also key. Implementing robust encryption, access control mechanisms, and adhering to stringent data protection regulations are critical aspects of sensitive data management. Building a secure environment that instills user confidence, protecting their privacy and intellectual property, establishes the foundation for a trustworthy AI-powered coding assistant.

L14. Testing and debugging is even harder: Testing and debugging LLM-based applications pose significant challenges due to their inherent complexity and the nuances of natural language processing. For instance, in the StackSpot AI, accurately retrieving and integrating diverse knowledge sources for enriching user prompts involves intricate semantic searches and translation processes. Verifying the fidelity of these retrieved chunks and the subsequent responses is far from trivial. Moreover, crafting prompts that not only trigger appropriate responses but also align seamlessly with user intent requires careful consideration. While the team initiated efforts in this direction, resolving this issue necessitates the implementation of a more robust testing framework.

6 DISCUSSION

In this section, we provide additional discussion based on our own learning.

6.1 Revisiting Lessons

In terms of **LLM-based lessons**, we noted significant differences between, say, traditional software development and LLM-based software development. For instance, when prompting an existing LLM, developers have to take care of the total tokens provided. However, knowing in advance the total number of tokens is not always possible, because the total number of tokens in a request also considers the LLM response, which is non-deterministic. Handling this scenario requires sophisticated error-handling mechanisms not often adopted in traditional software development.

Since we decided to use `gpt-3.5-turbo`, with a maximum of 4,097 tokens per request, the team had to reflect on several prompt optimization techniques (L1). This is particularly relevant given we enrich the prompt with the appropriate knowledge source (L3), and also with past conversations. Therefore, the size of the prompt would

endlessly increase, if the team did not take any counter-measurement approaches. Examples of a counter-measurement approach to preventing prompt overflow are chunking knowledge sources (L2) or reducing the number of messages stored in the chat history (L5). These approaches are also not definitive and can vary from context to context. For instance, the team explored several approaches for chunking data, from employing signal processing techniques to exploring software slicing techniques.

Still, knowing that the quality of the knowledge source could directly impact the quality of the LLM response, another important concern is about retrieving relevant knowledge sources (L3). The team explored different approaches, varying from different semantic search techniques. Observed results indicated that different approaches retrieved similar knowledge sources.

For **user-based lessons**, developing AI-based systems is non-trivial [9]. While a lot of effort has been put into developing and improving machine-learning algorithms, and selecting the correct training data, among other aspects [11], only somewhat recently the human-interface community has started paying attention to how these systems should be designed to interact with humans. In 2019, for instance, Amershi et al. [1] proposed a set of guidelines for human-ai interaction that has been adopted at Microsoft. These guidelines are called HAX. In this context, our lesson L10 (Allow Customization) is related to guideline 13 (Learn from user behavior) which suggests that an AI system should: “Personalize the user’s experience by learning from their actions over time.” Meanwhile, our lesson about L9 (Gather Feedback, Learn and Adapt to it) can be mapped to HAX guideline number 4 (Show contextually relevant information), i.e., StackSpot AI should, and indeed, does “Display information relevant to the user’s current task and environment.” As for the other User-based lessons (L7, L8, and L11), we did not find a direct mapping between them and the HAX guidelines. A possible explanation is that the guidelines focus solely on user interaction, while these three lessons refer to the aspects that StackSpot AI should support to be successful.

Finally, the **technical lessons** we observed are similar to other recommendations for building large-scale software systems. For instance, Twitter is likely to face the same trade-off between scalability and latency (L11) when computing sets of tweets to be recommended for each user. Meanwhile, GitHub must adopt good practices for sensitive data management (L12) since it handles code from individuals, open-source communities and private organizations. Arguably, AI-based coding assistants must be able to handle both aspects described in these lessons but, this requires further investigation. Since StackSpot AI is in its early periods of adoption, we do not have enough data to confirm this.

It is important to notice how different design decisions impact other design decisions, which in our case, means that one lesson learned led to another lesson. For instance, the need to manage tokens (L1) requires the management of chat history (L5), as well as the need to break knowledge sources into chunks (L2). Similarly, ensuring good responses (L7) is affected by the need to find a balance between scalability and latency (L12). This suggests that there is a lot to be learnt about the process of creating LLM-based applications.

6.2 Experimentation is Needed

Through our accumulated experience and the lessons we have gained, it became evident that many of the challenges encountered demanded a form of experimentation. This was especially true when determining the optimal chunking approach or devising effective methods for managing past conversations. In this domain, there is not a strict dichotomy between right and wrong answers; instead, solutions exist along a nuanced spectrum. This nuanced context creates an ideal scenario for collaboration between software researchers and engineers. For instance, seasoned researchers can support software engineers by guiding them through the curation and execution of more systematic software experiments. Conversely, engineers can assist researchers in translating their experiments into practical software features, bridging the gap between theoretical exploration and tangible, user-focused applications. This symbiotic relationship between experimentation and implementation forms the bedrock of our iterative software development process.

7 RELATED WORK

As mentioned before, we are witnessing a race for AI-based code assistants, also called LLMs for code, with new tools being launched every couple of months. Research on this topic is not different with new papers being published at all major conferences. While unofficial evidence suggests these tools are impacting developers’ workflows⁶, research on this topic is still inconclusive [30]. Furthermore, most research has been conducted as controlled laboratory experiments instead of collecting data from the field. Despite that, the results are encouraging. For instance, even when AI-based code assistants present bad suggestions, these can be beneficial as they provide a “starting point” for developers. This is true both in the context of code creation [24] and code translation [28]. One of the few exceptions for non-laboratory experiments is Ziegler’s et al. [34] who proposed a set of metrics to study the impact of AI-based code assistants on developers’ self-reported productivity.

These previous studies focus on the *usage* of AI-based assistants. Research on the challenges associated with *building* them is even more scarce. To the best of our knowledge, one of the exceptions is Xu’s et al. [30] who built a “hybrid code generation and code retrieval plugin for the Python PyCharm IDE, which takes as input natural language queries” to study the impact of code generation and retrieval techniques in developers’ productivity. However, Xu and colleagues did *not* use a large language model, instead for code retrieval they implemented a wrapper around a general-purpose search engine, while for code generation they used a previous implementation of a tree-based semantic parsing mode.

8 LIMITATIONS

As with any empirical work, this one also has many limitations and threats to validity. The limitations of this work include the scope of the research and the generalizability of the findings.

Firstly, this work is based on the observation of a single software development team, marking their inaugural venture into building an LLM-based application. Consequently, some lessons identified may stem from their lack of prior technical experience in the AI-applied domain. Additionally, the observed lessons elucidated in this paper

⁶<https://ai.googleblog.com/2022/07/ml-enhanced-code-completion-improves.html>

are specific to the development of StackSpot AI, an LLM-based AI coding assistant. Our study focused exclusively on this application, potentially constraining the broader applicability of our findings to other types of LLM-based applications. The challenges pinpointed might not comprehensively capture the intricacies faced by diverse projects across varied domains.

Secondly, the study's reliance on a single dataset and a specific group of software engineers might introduce bias and restrict the overall perspective. Additionally, the rapidly evolving nature of AI technologies implies that some challenges identified during the study might become obsolete as newer models and techniques emerge.

Furthermore, the study primarily concentrated on the process of building StackSpot AI, potentially overlooking other crucial factors such as ethical considerations or user adoption. Lastly, the qualitative nature of our research might limit the ability to quantify and statistically analyze the challenges identified, leading to a qualitative bias in our conclusions.

These limitations highlight the need for future studies to encompass a wider array of LLM-based applications and incorporate diverse perspectives to offer a more comprehensive understanding of the challenges faced when building applications in this domain.

9 CONCLUSIONS & FUTURE WORK

In this paper, we delved into the intricate landscape of LLM-based applications, focusing on the creation of an AI-based coding assistant called StackSpot AI. We outlined the challenges faced in this endeavour, exploring diverse strategies, including retrieval-based techniques and contextual enrichment in order to improve response quality, ensuring a more nuanced understanding of user queries and generating precise, contextually relevant answers. Through developer interviews and issue tracker analysis, we curated 14 lessons, illuminating crucial aspects of LLM-based application development.

Our most significant findings revolved around these 12 lessons that can be broadly classified into LLM-based, user-based, and technical. The technical lessons are somewhat similar to others faced in large-scale software systems. The user-based lessons are somewhat similar to those faced in the development of human-AI interactions, although we identified challenges (L7, L8 and L11) that have not been reported in the literature. Finally, we argue the LLM-based lessons (L1-L6) are our biggest contributions because they are singular to the process of creating an LLM-based application for software development.

In the future, we plan to expand our research agenda on this topic. First, we plan to investigate advanced techniques for even more precise retrieval, studying diverse contextual enrichment methods, and delving into user experience optimization can enhance the capabilities of LLM-based AI coding assistants. Additionally, we aim at fostering collaboration between researchers and industry professionals could lead to more robust, user-centric applications, propelling the field forward and empowering developers worldwide.

ACKNOWLEDGMENTS

We would like to thank the reviewers for their valuable comments. This work is partially supported by INES (www.ines.org.br), CNPq

(grants 420406/2023-9, 442779/2023-2, 465614/2014-0, and 308623/2022-3), FAPESP (053/2021), FACEPE (grants APQ-0399-1.03/17, PRONEX APQ/ 0388-1.03/14), and CAPES (88887.136410/2017-00).

REFERENCES

- [1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *CHI 2019*. ACM. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/> CHI 2019 Honorable Mention Award.
- [2] Samuel R Bowman. 2023. Eight things to know about large language models. *arXiv preprint arXiv:2304.00612* (2023).
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *arXiv:2005.14165* [cs.CL]
- [4] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Kaijie Zhu, Hao Chen, Linyi Yang, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2023. A survey on evaluation of large language models. *arXiv preprint arXiv:2307.03109* (2023).
- [5] Victoria Clarke and Virginia Braun. 2013. *Successful Qualitative Research: A Practical Guide for Beginners*. Sage, London.
- [6] Mohamad Diab, Julian Herrera, and Bob Chernow. 2022. *Stable Diffusion Prompt Book*. Accessed: 2023-10-01.
- [7] Jean-Baptiste Döderlein, Mathieu Acher, Djamel Eddine Khelladi, and Benoit Combemale. 2023. Piloting Copilot and Codex: Hot Temperature, Cold Prompts, or Black Magic? *arXiv:2210.14699* [cs.SE]
- [8] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217* (2023).
- [9] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. *arXiv preprint arXiv:2310.03533* (2023).
- [10] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.
- [11] Gökrem Giray. 2021. A software engineering perspective on engineering machine learning systems: State of the art and challenges. *Journal of Systems and Software* 180 (2021), 111031.
- [12] Jingxuan He and Martin T. Vechev. 2023. Controlling Large Language Models to Generate Secure and Vulnerable Code. *CoRR* abs/2302.05319 (2023). <https://doi.org/10.48550/arXiv.2302.05319> arXiv:2302.05319
- [13] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John C. Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *CoRR* abs/2308.10620 (2023). <https://doi.org/10.48550/arXiv.2308.10620> arXiv:2308.10620
- [14] CS Krishna. 2023. Prompt Generate Train (PGT): A framework for few-shot domain adaptation, alignment, and uncertainty calibration of a retriever augmented generation (RAG) model for domain specific open book question-answering. *arXiv preprint arXiv:2307.05915* (2023).
- [15] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [16] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. AceCoder: Utilizing Existing Code to Enhance Code Generation. *arXiv:2303.17780* [cs.SE]
- [17] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *CoRR* abs/2307.03172 (2023). <https://doi.org/10.48550/arXiv.2307.03172> arXiv:2307.03172
- [19] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2023. LLM is Like a Box of Chocolates: the Non-determinism of ChatGPT in Code Generation. *CoRR* abs/2308.02828 (2023). <https://doi.org/10.48550/arXiv.2308.02828> arXiv:2308.02828

- [20] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.
- [21] Gustavo Pinto, Cleidson de Souza, Thayssa Rocha, Igor Steinmacher, Alberto de Souza, and Edward Monteiro. 2023. Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes. *arXiv:2311.18452* [cs.SE]
- [22] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces* (Sydney, NSW, Australia) (*IUI '23*). Association for Computing Machinery, New York, NY, USA, 491–514. <https://doi.org/10.1145/3581641.3584037>
- [23] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language models in medicine. *Nature medicine* (2023), 1–11.
- [24] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (*CHI EA '22*). Association for Computing Machinery, New York, NY, USA, Article 332, 7 pages. <https://doi.org/10.1145/3491101.3519665>
- [25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (*NIPS'17*). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [26] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2023. Software testing with large language model: Survey, landscape, and vision. *arXiv preprint arXiv:2307.07221* (2023).
- [27] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv:2201.11903* [cs.CL]
- [28] Justin D. Weisz, Michael Muller, Stephanie Houde, John Richards, Steven I. Ross, Fernando Martinez, Mayank Agarwal, and Kartik Talamadupula. 2021. Perfection Not Required? Human-AI Partnerships in Code Translation. In *26th International Conference on Intelligent User Interfaces* (College Station, TX, USA) (*IUI '21*). Association for Computing Machinery, New York, NY, USA, 402–412. <https://doi.org/10.1145/3397481.3450656>
- [29] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhajan Kambadur, David Rosenberg, and Gideon Mann. 2023. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564* (2023).
- [30] Frank F. Xu, Bogdan Vasilescu, and Graham Neubig. 2022. In-IDE Code Generation from Natural Language: Promise and Challenges. *ACM Trans. Softw. Eng. Methodol.* 31, 2, Article 29 (mar 2022), 47 pages. <https://doi.org/10.1145/3487569>
- [31] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Bing Yin, and Xia Hu. 2023. Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond. *CoRR* abs/2304.13712 (2023). <https://doi.org/10.48550/arXiv.2304.13712> *arXiv:2304.13712*
- [32] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, DongGyun Han, and David Lo. 2023. What Do Code Models Memorize? An Empirical Study on Large Language Models of Code. *CoRR* abs/2308.09932 (2023). <https://doi.org/10.48550/arXiv.2308.09932> *arXiv:2308.09932*
- [33] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Evaluations on HumanEval-X. *arXiv:2303.17568* [cs.LG]
- [34] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity Assessment of Neural Code Completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (San Diego, CA, USA) (*MAPS 2022*). Association for Computing Machinery, New York, NY, USA, 21–29. <https://doi.org/10.1145/3520312.3534864>